

GESTURE CONTROLLED VIRTUAL MOUSE BASED ON HUMAN COMPUTER INTERACTION INTEGRATED WITH VOICE COMMANDS

(PROJECT PHASE- I)
*submitted in partial fulfillment of the requirements
for the award of the degree in*

**BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING**

by

**S. KARTHICK RAJA (Reg.No:211061101201)
A.R. KAILASH (Reg.No:211061101184)
C.KAVIARASAN (Reg.No:211061101207)**



Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY
University with Graded Autonomy Status



(An ISO 21001 : 2018 Certified Institution)
Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.

**DEPARTMENT
OF
COMPUTER SCIENCE AND ENGINEERING**

NOVEMBER 2024



Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY

University with Graded Autonomy Status

(An ISO 21001 : 2018 Certified Institution)

Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this Project Report (Project Phase-I) is the bonafide work of

Mr. S. KARTHICK RAJA

Reg. No 211061101201,

Mr. A.R. KAILASH

Reg. No 211061101184,

Mr. C. KAVIARASAN

Reg. No 211061101207, who carried out the project

entitled “**GESTURE CONTROLLED VIRTUAL MOUSE BASED ON HUMAN COMPUTER INTERACTION INTEGRATED WITH VOICE COMMANDS**” under our supervision from June 2024 to November 2024.

Internal Guide

Mr.D.SYED ALI
Add HOD of CFIS

Project Coordinator

Dr.K.S.RAMANUJAM
Assistant professor
Mr.D.SYED ALI
Add HOD of CFIS

Department Head

Dr.S.Geetha
HOD of CSE

Department of CSE,
Dr .M.G.R.Educational
and Research Institute
Deemed to be University

Department of CSE
Dr.M.G.R Educational
and Research Institute
Deemed to be University

Department of CSE
Dr.M.G.R Educational
and Research Institute
Deemed to be University

Submitted for Viva Voce Examination held on _____

Internal Examiner

External Examiner



Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY

University with Graded Autonomy Status

(An ISO 21001 : 2018 Certified Institution)

Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.



DECLARATION

We **S. KARTHICK RAJA (211061101201), A.R. KAILASH (211061101184), C. KAVIARASAN (211061101207)**, hereby declare that the Project Report (Project Phase-I) entitled **“GESTURE CONTROLLED VIRTUAL MOUSE BASED ON HUMAN COMPUTER INTERACTION INTEGRATED WITH VOICE COMMANDS”** is done by us under the guidance of **Mr. D.SYED ALI** is submitted in partial fulfillment of the requirements for the award of the degree in **BACHELOR OF TECHNOLOGY** in Computer Science and Engineering.

DATE:

PLACE:

1.

2.

3.

SIGNATURE OF THE CANDIDATE(S)

ACKNOWLEDGEMENT

We would first like to thank our beloved Founder Respected Chancellor **Thiru.Dr.A.C. SHANMUGAM, B.A., B.L.**, President **Er. A.C.S.Arunkumar, B.Tech., M.B.A.**, and Secretary **Thiru A.RAVIKUMAR** for all the encouragement and support extended to us during the tenure of this project and also our years of studies in his wonderful University.

We express my heartfelt thanks to our Vice Chancellor **Prof. Dr. S. GEETHALAKSHMI** in providing all the support of my Project (Project Phase-I).

We express my heartfelt thanks to our Head of the Department, **Prof. Dr. S.Geetha**, who has been actively involved and very influential from the start till the completion of our project.

Our sincere thanks to our Project Coordinators **Mr.D.SYED ALI, Dr.K.S.RAMANUJAM**, and Project guide **Mr.D.SYED ALI** for their continuous guidance and encouragement throughout this work, which has made the project a success.

We would also like to thank all the teaching and non-teaching **Mr.D.SYED ALI** staffs of Computer Science and Engineering department, for their constant support and the encouragement given to us while we went about to achieving my project goals.

TABLE OF CONTENT

S.NO	TITLE	PAGE NUMBER
	LIST OF THE FIGURES	I
	LIST OF ABBREVIATIONS	I
1	ABSTRACT	1
2	CHAPETER 1: INTRODUCTION 1.1 Background 1.2 Objective 1.3 Scope of the project	2
3	CHAPETER 2: REQUIREMENT ANALYSIS 2.1 Literature Survey 2.2 User requirements 2.3 System requirements	3-4
4	CHAPETER 3: REQUIREMENT SPECIFICATION 3.1 User & system requiremnts 3.1.1 User Requirements 3.1.2 System Requirements 3.1.2.1 Hardware Requirements 3.1.2.2 Software Requirements 3.2 Development Requirements 3.3 Runtime Requirements 3.3.1 Installation Requirements	5-6
5	CHAPETER 4:DESIGN 4.1 System Architecture 4.2 Workflow Diagrams	7-9
6	CHAPETER 5: IMPLEMENTATION 5.1 Code Implementation 5.2 Testing 5.3 Gesture Detection Output	10-20
7	CHAPETER 6: CONCLUSION	21
8	REFERENCES/BIBILOGRAPHY	22

LIST OF THE FIGURES

S.NO	FIGURE NUMBER	NAME OF THE FIGURE	PAGE NUMBER
1	4.2.1	System Workflow of Gesture Recognition	8
2	4.2.2	System Workflow of Voice Commands	9
3	5.3.1	Initial pop-up window	16
4	5.3.2	Neutral Gesture	17
5	5.3.3	Move Cursor	17
6	5.3.4	Left Click	18
7	5.3.5	Right Click	18
8	5.3.6	Double Click	19
9	5.3.7	Drag and Drop	19
10	5.3.8	Multiple Item Selection	20

LIST OF ABBREVIATIONS

- 1.OPEN CV** - Open Computer Vision library
- 2.RAM** – Random Access Memory
- 3.UI** – User Interface
- 4.IDE** - Integrated Development Environment
- 5.GUI** – Graphical User Interface
- 6.HCI** – Human Computer Interaction
- 7.pyttsx3** - Python Text-to-Speech version 3

ABSTRACT

In this project, we propose a **Gesture Controlled Virtual Mouse** that enhances Human-Computer Interaction by integrating hand gestures and voice commands. This system enables users to perform mouse operations such as clicking, scrolling, and selecting without physical contact, utilizing only gestures and voice inputs. By using advanced Machine Learning and Computer Vision techniques, the project offers an efficient, user-friendly, and hygienic alternative to traditional input devices. Key components include OpenCV for image processing, MediaPipe for hand tracking, and Pyttsx3 for text-to-speech conversion. The project is intended for Windows environments and aims to promote accessibility and convenience in human-computer interactions.

CHAPTER 1 - INTRODUCTION

1.1 BACKGROUND:

The rise of touchless technology has led to innovative methods for interacting with computers. Traditional input methods such as keyboards and mice are increasingly complemented by gesture and voice-based interfaces. Our project, a Gesture Controlled Virtual Mouse, aims to bring these new-age interaction methods into everyday computing, reducing the need for physical contact.

1.2 OBJECTIVE:

This project's objective is to create a gesture-controlled virtual mouse that can control a computer system through hand gestures and voice commands. By minimizing physical interaction with hardware, the project seeks to improve usability, hygiene, and accessibility.

1.3 SCOPE OF THE PROJECT:

The scope of this project includes developing a functional prototype that can accurately detect gestures and respond to voice commands for executing mouse actions. This system is implemented using a standard webcam, allowing accessibility without additional hardware.

CHAPTER 2 - REQUIREMENT ANALYSIS

2.1 Literature Survey

2.1.1 Deep Learning-Based Real-Time AI Virtual Mouse System

A study published in Journal of Healthcare Engineering by Hasan et al. (2021) introduced an AI-powered virtual mouse system using computer vision to avoid the spread of COVID-19. The system employs deep learning for gesture recognition, enabling users to perform basic mouse functions without physical contact. This study highlights the use of computer vision in HCI, promoting hygienic interaction, especially relevant during the pandemic.

2.1.2 Voice Enabled Personal Assistant for PCs

In 2021, V. Geetha and colleagues proposed a voice-enabled personal assistant using Python, as documented in the International Journal of Engineering and Advanced Technology. The assistant integrates voice commands to perform tasks on a computer, enhancing accessibility for users. This research underscores the utility of voice control in HCI, setting a foundation for integrating voice commands into our project.

2.1.3 Virtual Mouse Using Hand Gesture

Chaithanya et al. (2018) developed a virtual mouse that uses color-based gesture recognition to control cursor movements, as presented in the International Research Journal of Engineering and Technology. Their model detects color bands on fingertips to perform actions like clicking and scrolling. Although effective, the dependency on physical color markers limits usability. This project addresses these limitations by using MediaPipe for markerless hand tracking.

2.2 User Requirements

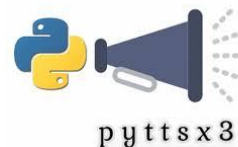
The system should be capable of:

- Recognizing hand gestures for basic mouse functions.
- Responding accurately to voice commands.
- Operating with minimal latency to provide a smooth user experience.

2.3 System Requirements

To function optimally, the system requires:

- A standard webcam for real-time video input.
- A computer with at least **4GB RAM** and a **3.4 GHz Processor**.
- Libraries including OpenCV, MediaPipe, PyAutoGUI, and Pyttsx3.



CHAPTER 3 - REQUIREMENT SPECIFICATION

3.1 User & System Requirements

3.1.1 User Requirements

This section outlines what end-users expect from the gesture-controlled virtual mouse system

- **Touchless Interaction:** Users should perform basic mouse functions (clicking, scrolling, dragging) without physical contact.
- **Responsive Gestures:** The system must recognize gestures quickly and accurately, with minimal delay.
- **Voice Command Integration:** Users should issue basic voice commands for tasks like opening apps, controlling volume, and shutting down.
- **Compatibility:** The system should seamlessly integrate with existing applications, requiring no extensive setup.

3.1.2 System Requirements

This section details the essential hardware and software needed for system development

3.1.2.1 Hardware Requirements:

- **Webcam:** A standard webcam (minimum 720p resolution) for accurate gesture tracking.
- **Processor:** At least 3.4 GHz to handle real-time image processing.
- **Memory:** Minimum of 4GB RAM for smooth operation.

3.1.2.2 Software Requirements:

- **Operating System:** Windows 10 or higher.
- **Programming Language:** Python 3.7 or above, supporting necessary libraries for machine learning.
- **Libraries:**
 - **OpenCV:** For image processing and gesture detection.
 - **MediaPipe:** For hand-tracking functionality.
 - **PyAutoGUI:** To translate gestures into mouse actions.
 - **Pyttsx3:** For text-to-speech in the voice command module.

3.2 Development Requirements

The environment used for coding, testing, and debugging includes:

- **IDE:** Visual Studio Code or PyCharm, providing support for Python and its libraries.
- **Library Installation:** Required libraries (OpenCV, MediaPipe, PyAutoGUI, Pyttsx3) installed via pip.
- **Version Control:** Git for version control, ensuring code backups and enabling collaboration.
- **Testing Framework:** PyTest or Unittest for testing gesture recognition and voice command features.

3.3 Runtime Requirements

To run the application, specific configurations and settings are essential:

- **Webcam Setup:** A properly positioned webcam for tracking hand movements.
- **Processor and RAM:** Minimum of 4GB RAM and a 3.4 GHz processor for effective real-time video processing.
- **Python Dependencies:** All necessary libraries must be installed and verified for compatibility.
- **User Interface (UI):** A minimal UI should display active gestures or commands recognized to assist user interaction.

3.3.1 Installation Requirements

Steps and tools for installing and configuring the system:

1. **Python Installation:** Install Python 3.7 or above.
2. **Library Installation:** Install required libraries using pip (e.g., pip install opencv-python mediapipe pyautogui pyttsx3).
3. **Configuration File:** Set up a configuration file or script for initial webcam and system settings.

CHAPTER 4 - DESIGN

4.1 System Architecture :

The system architecture is divided into two modules **Gesture Recognition** and **Voice Command Integration**.

- Gesture recognition uses MediaPipe for tracking hand movements and OpenCV for processing video input.
- Voice commands are processed using Pyttsx3 for text-to-speech conversion.

4.2 Workflow Diagrams



Fig. 4.2.1: System Workflow of Gesture Recognition

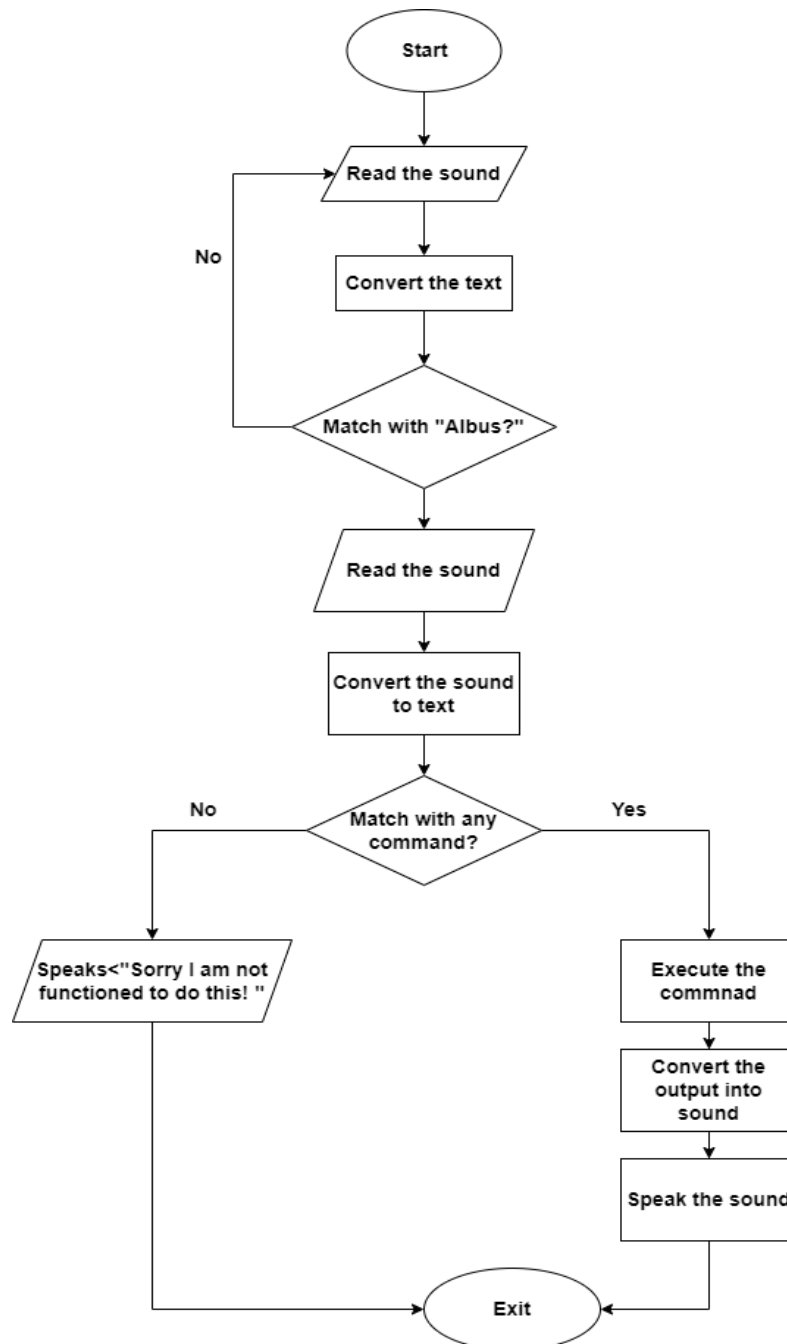


Fig. 4.2.2: System Workflow of Voice Commands.

CHAPTER 5 - IMPLEMENTATION

5.1 Code Implementation:

The project is coded in Python. Key modules include:

- **Gesture Recognition** using MediaPipe.
- **Mouse Control** using PyAutoGUI.
- **Voice Commands** implemented with Pyttsx3.

5.2 Testing:

Testing was conducted in a controlled environment to ensure accuracy in hand gesture detection and command response.

File Name: Gesture_Controller.py

```
1  # Imports
2
3  import cv2
4  import mediapipe as mp
5  import pyautogui
6  import math
7  from enum import IntEnum
8  from ctypes import cast, POINTER
9  from comtypes import CLSCTX_ALL
10 from pycaw.pycaw import AudioUtilities, IAudioEndpointVolume
11 from google.protobuf.json_format import MessageToDict
12 import screen_brightness_control as sbcontrol
13
14 pyautogui.FAILSAFE = False
15 mp_drawing = mp.solutions.drawing_utils
16 mp_hands = mp.solutions.hands
17
18 # Gesture Encodings
19 class Gest(IntEnum):
20     # Binary Encoded
21     """
22     Enum for mapping all hand gesture to binary number.
23     """
24
25     FIST = 0
26     PINKY = 1
27     RING = 2
28     MID = 4
29     LAST3 = 7
30     INDEX = 8
31     FIRST2 = 12
32     LAST4 = 15
33     THUMB = 16
34     PALM = 31
35
36     # Extra Mappings
37     V_GEST = 33
```

```

36 # Extra Mappings
37 V_GEST = 33
38 TWO_FINGER_CLOSED = 34
39 PINCH_MAJOR = 35
40 PINCH_MINOR = 36
41
42 # Multi-handedness Labels
43 class HLabel(IntEnum):
44     MINOR = 0
45     MAJOR = 1
46
47 # Convert Mediapipe Landmarks to recognizable Gestures
48 class HandRecog:
49     """
50     Convert Mediapipe Landmarks to recognizable Gestures.
51     """
52
53     def __init__(self, hand_label):
54         """
55         Constructs all the necessary attributes for the HandRecog object.
56
57         Parameters
58         -----
59         finger : int
60             Represent gesture corresponding to Enum 'Gest',
61             stores computed gesture for current frame.
62         ori_gesture : int
63             Represent gesture corresponding to Enum 'Gest',
64             stores gesture being used.
65         prev_gesture : int
66             Represent gesture corresponding to Enum 'Gest',
67             stores gesture computed for previous frame.
68         frame_count : int
69             total no. of frames since 'ori_gesture' is updated.
70         hand_result : Object
71             Landmarks obtained from mediapipe.
72         hand_label : int
73             Represents multi-handedness corresponding to Enum 'HLabel'.
74         """
75
76         self.finger = 0
77         self.ori_gesture = Gest.PALM
78         self.prev_gesture = Gest.PALM
79         self.frame_count = 0
80         self.hand_result = None
81         self.hand_label = hand_label
82
83     def update_hand_result(self, hand_result):
84         self.hand_result = hand_result
85
86     def get_signed_dist(self, point):
87         """
88         returns signed euclidean distance between 'point'.
89
90         Parameters
91         -----
92         point : list containing two elements of type list/tuple which represents
93             landmark point.
94
95         Returns
96         -----
97         float
98         """
99         sign = -1
100         if self.hand_result.landmark[point[0]].y < self.hand_result.landmark[point[1]].y:
101             sign = 1
102         dist = (self.hand_result.landmark[point[0]].x - self.hand_result.landmark[point[1]].x)**2

```

```

103     dist += (self.hand_result.landmark[point[0]].y - self.hand_result.landmark[point[1]].y)**2
104     dist = math.sqrt(dist)
105     return dist*sign
106
107 def get_dist(self, point):
108     """
109     returns euclidean distance between 'point'.
110
111     Parameters
112     -----
113     point : list containing two elements of type list/tuple which represents
114            landmark point.
115
116     Returns
117     -----
118     float
119     """
120     dist = (self.hand_result.landmark[point[0]].x - self.hand_result.landmark[point[1]].x)**2
121     dist += (self.hand_result.landmark[point[0]].y - self.hand_result.landmark[point[1]].y)**2
122     dist = math.sqrt(dist)
123     return dist
124
125 def get_dz(self, point):
126     """
127     returns absolute difference on z-axis between 'point'.
128
129     Parameters
130     -----
131     point : list containing two elements of type list/tuple which represents
132            landmark point.
133
134     Returns
135     -----
136     float
137     """
138     return abs(self.hand_result.landmark[point[0]].z - self.hand_result.landmark[point[1]].z)
139
140 # Function to find Gesture Encoding using current finger_state.
141 # Finger_state: 1 if finger is open, else 0
142 def set_finger_state(self):
143     """
144     set 'finger' by computing ratio of distance between finger tip
145     , middle knuckle, base knuckle.
146
147     Returns
148     -----
149     None
150     """
151     if self.hand_result == None:
152         return
153
154     points = [[8,5,0],[12,9,0],[16,13,0],[20,17,0]]
155     self.finger = 0
156     self.finger = self.finger | 0 #thumb
157     for idx, point in enumerate(points):
158
159         dist = self.get_signed_dist(point[:2])
160         dist2 = self.get_signed_dist(point[1:])
161
162         try:
163             ratio = round(dist/dist2,1)
164         except:
165             ratio = round(dist/0.01,1)
166
167         self.finger = self.finger << 1
168         if ratio > 0.5 :
169             self.finger = self.finger | 1
170
171

```

```

173 def get_gesture(self):
174     """
175     returns int representing gesture corresponding to Enum 'Gest'.
176     sets 'frame_count', 'ori_gesture', 'prev_gesture',
177     handles fluctuations due to noise.
178
179     Returns
180     -----
181     int
182     """
183     if self.hand_result == None:
184         return Gest.PALM
185
186     current_gesture = Gest.PALM
187     if self.finger in [Gest.LAST3, Gest.LAST4] and self.get_dist([8,4]) < 0.05:
188         if self.hand_label == HLabel.MINOR :
189             current_gesture = Gest.PINCH_MINOR
190         else:
191             current_gesture = Gest.PINCH_MAJOR
192
193     elif Gest.FIRST2 == self.finger :
194         point = [[8,12],[5,9]]
195         dist1 = self.get_dist(point[0])
196         dist2 = self.get_dist(point[1])
197         ratio = dist1/dist2
198         if ratio > 1.7:
199             current_gesture = Gest.V_GEST
200         else:
201             if self.get_dz([8,12]) < 0.1:
202                 current_gesture = Gest.TWO_FINGER_CLOSED
203             else:
204                 current_gesture = Gest.MID
205
206     else:
207         current_gesture = self.finger
208
209 # Locate Hand to get Cursor Position
210 # Stabilize cursor by Dampening
211 def get_position(hand_result):
212     """
213     returns coordinates of current hand position.
214
215     Locates hand to get cursor position also stabilize cursor by
216     dampening jerky motion of hand.
217
218     Returns
219     -----
220     tuple(float, float)
221     """
222     point = 9
223     position = [hand_result.landmark[point].x ,hand_result.landmark[point].y]
224     sx,sy = pyautogui.size()
225     x_old,y_old = pyautogui.position()
226     x = int(position[0]*sx)
227     y = int(position[1]*sy)
228     if Controller.prev_hand is None:
229         Controller.prev_hand = x,y
230     delta_x = x - Controller.prev_hand[0]
231     delta_y = y - Controller.prev_hand[1]
232
233     distsq = delta_x**2 + delta_y**2
234     ratio = 1
235     Controller.prev_hand = [x,y]
236
237     if distsq <= 25:
238         ratio = 0
239     elif distsq <= 900:
240         ratio = 0.07 * (distsq ** (1/2))
241     else:
242         ratio = 2.1
243     x , y = x_old + delta_x*ratio , y_old + delta_y*ratio
244     return (x,y)

```

```

471 ----- Main Class -----
472 Entry point of Gesture Controller
473 ...
474
475
476
477 class GestureController:
478     """
479     Handles camera, obtain landmarks from mediapipe, entry point
480     for whole program.
481
482     Attributes
483     -----
484     gc_mode : int
485         indicates weather gesture controller is running or not,
486         1 if running, otherwise 0.
487     cap : Object
488         object obtained from cv2, for capturing video frame.
489     CAM_HEIGHT : int
490         highet in pixels of obtained frame from camera.
491     CAM_WIDTH : int
492         width in pixels of obtained frame from camera.
493     hr_major : Object of 'HandRecog'
494         object representing major hand.
495     hr_minor : Object of 'HandRecog'
496         object representing minor hand.
497     dom_hand : bool
498         True if right hand is domaniant hand, otherwise False.
499         default True.
500     """
501     gc_mode = 0
502     cap = None
503     CAM_HEIGHT = None
504     CAM_WIDTH = None
505     hr_major = None # Right Hand by default
506     hr_minor = None # Left hand by default
507     dom_hand = True
508
509     def __init__(self):
510         """Initilaizes attributes."""
511         GestureController.gc_mode = 1
512         GestureController.cap = cv2.VideoCapture(0)
513         GestureController.CAM_HEIGHT = GestureController.cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
514         GestureController.CAM_WIDTH = GestureController.cap.get(cv2.CAP_PROP_FRAME_WIDTH)
515
516     def classify_hands(results):
517         """
518         sets 'hr_major', 'hr_minor' based on classification(left, right) of
519         hand obtained from mediapipe, uses 'dom_hand' to decide major and
520         minor hand.
521         """
522         left , right = None, None
523         try:
524             handedness_dict = MessageToDict(results.multi_handedness[0])
525             if handedness_dict['classification'][0]['label'] == 'Right':
526                 right = results.multi_hand_landmarks[0]
527             else :
528                 left = results.multi_hand_landmarks[0]
529         except:
530             pass
531
532         try:
533             handedness_dict = MessageToDict(results.multi_handedness[1])
534             if handedness_dict['classification'][0]['label'] == 'Right':
535                 right = results.multi_hand_landmarks[1]
536             else :
537                 left = results.multi_hand_landmarks[1]
538         except:
539             pass
540
541         if GestureController.dom_hand == True:
542             GestureController.hr_major = right
543             GestureController.hr_minor = left

```

```

544         else :
545             GestureController.hr_major = left
546             GestureController.hr_minor = right
547
548     def start(self):
549         """
550         Entry point of whole program, captures video frame and passes, obtains
551         landmark from mediapipe and passes it to 'handmajor' and 'handminor' for
552         controlling.
553         """
554
555         handmajor = HandRecog(HLabel.MAJOR)
556         handminor = HandRecog(HLabel.MINOR)
557
558         with mp_hands.Hands(max_num_hands = 2, min_detection_confidence=0.5, min_tracking_confidence=0.5) as hands:
559             while GestureController.cap.isOpened() and GestureController.gc_mode:
560                 success, image = GestureController.cap.read()
561
562                 if not success:
563                     print("Ignoring empty camera frame.")
564                     continue
565
566                 image = cv2.cvtColor(cv2.flip(image, 1), cv2.COLOR_BGR2RGB)
567                 image.flags.writeable = False
568                 results = hands.process(image)
569
570                 image.flags.writeable = True
571                 image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
572
573                 if results.multi_hand_landmarks:
574                     GestureController.classify_hands(results)
575                     handmajor.update_hand_result(GestureController.hr_major)
576                     handminor.update_hand_result(GestureController.hr_minor)
577
578                     handmajor.set_finger_state()
579                     handmajor.set_finger_state()
580                     handminor.set_finger_state()
581                     gest_name = handminor.get_gesture()
582
583                     if gest_name == Gest.PINCH_MINOR:
584                         Controller.handle_controls(gest_name, handminor.hand_result)
585                     else:
586                         gest_name = handmajor.get_gesture()
587                         Controller.handle_controls(gest_name, handmajor.hand_result)
588
589                     for hand_landmarks in results.multi_hand_landmarks:
590                         mp_drawing.draw_landmarks(image, hand_landmarks, mp_hands.HAND_CONNECTIONS)
591
592                 else:
593                     Controller.prev_hand = None
594                     cv2.imshow('Gesture Controller', image)
595                     if cv2.waitKey(5) & 0xFF == 13:
596                         break
597
598                 GestureController.cap.release()
599                 cv2.destroyAllWindows()
600
601 gc1 = GestureController()
602 gc1.start()

```

5.3 Gesture Detection Output:

- *Initial pop-up window*

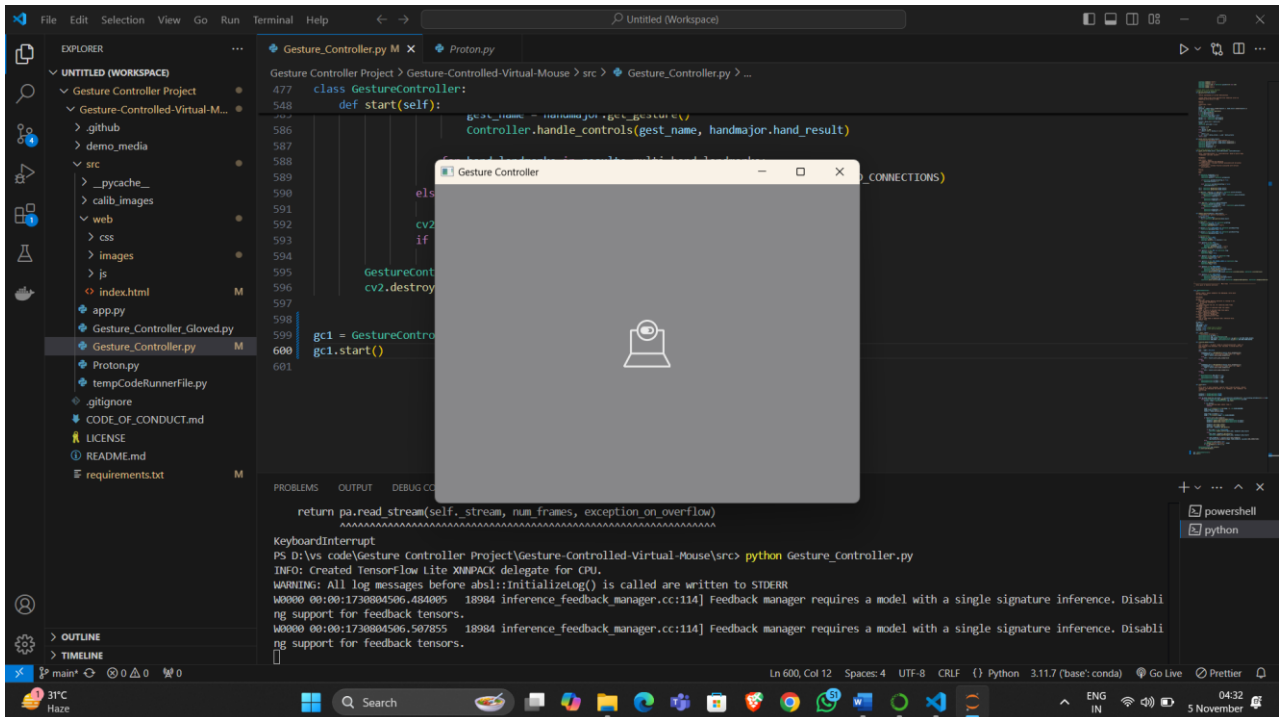


Fig.5.3.1: Initial pop-up window

- **Neutral Gesture**

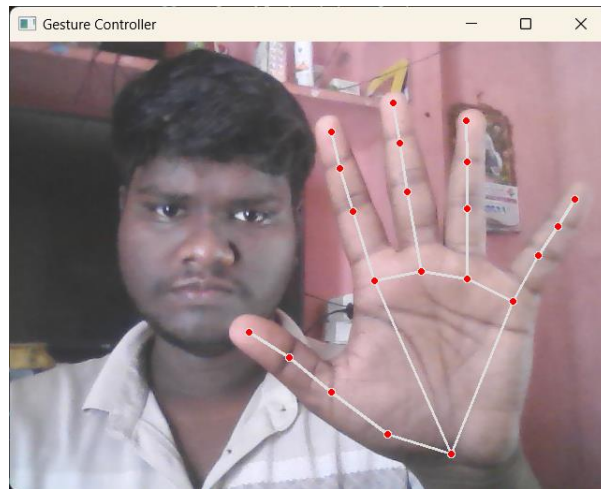


Fig.5.3.2: Neutral Gesture

Explanation: Neutral Gesture. Used to halt/stop execution of current gesture.

- **Move Cursor**



Fig.5.3.3: Move Cursor

Explanation: Cursor is assigned to the midpoint of index and middle fingertips. This gesture moves the cursor to the desired location. Speed of the cursor movement is proportional to the speed of hand.

- ***Left Click***

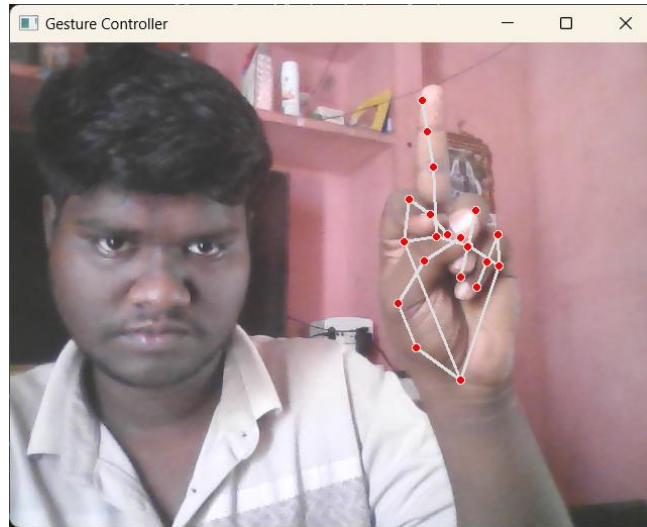


Fig.5.3.4: Left Click

Explanation: Gesture for single left click

- ***Right Click***

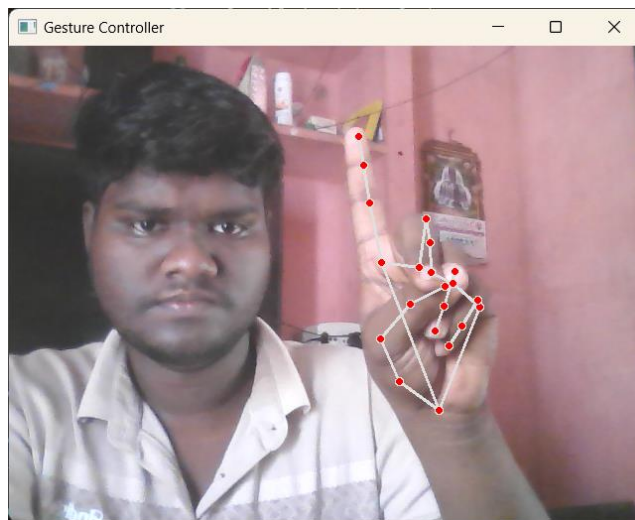


Fig.5.3.5: Right Click

Explanation: Gesture for single right click

- ***Double Click***

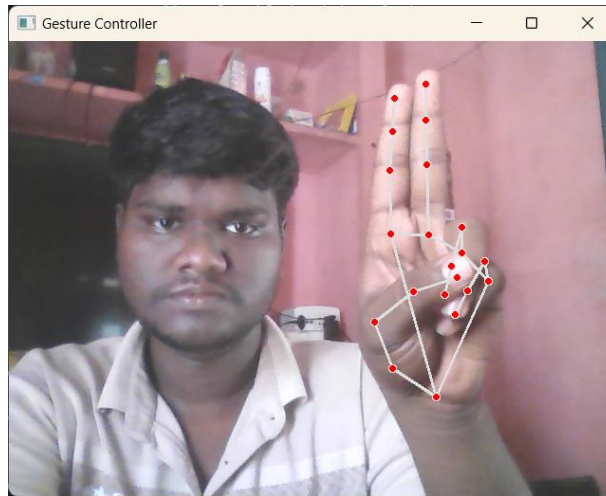


Fig.5.3.6: ***Double Click***

Explanation: Gesture for double click

- ***Drag and Drop***



Fig.5.3.7: Drag and Drop

Explanation: Gesture for drag and drop functionality. Can be used to move/tranfer files from one directory to other.

- ***Multiple Item Selection***

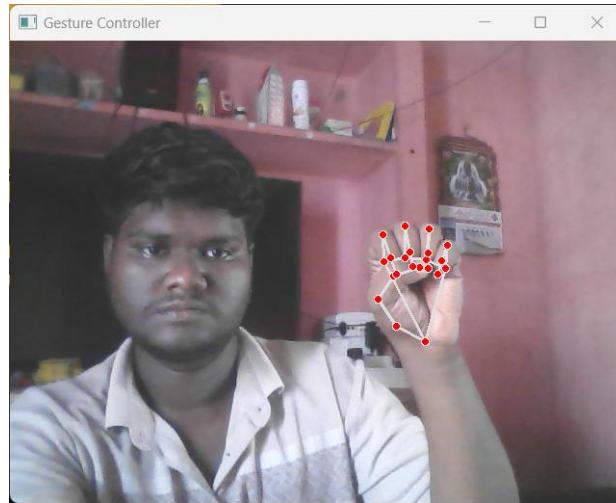


Fig.5.3.8: Multiple Item Selection

Explanation: Gesture to select multiple items

CHAPTER 6- CONCLUSION

In conclusion, this project presents a Gesture Controlled Virtual Mouse that allows touchless computer interaction using hand gestures and voice commands. By integrating Machine Learning and Computer Vision tools like MediaPipe and OpenCV, the system enables essential mouse functions—clicking, scrolling, and selection—without additional hardware, enhancing accessibility and hygiene. The project addresses the limitations of traditional input devices, offering a user-friendly, cost-effective solution suited for shared or public spaces. Although limitations such as sensitivity to lighting and reliance on clear voice input were identified, future improvements could expand the gesture set, adapt to various lighting conditions, and refine voice recognition. This project lays a foundation for accessible and innovative HCI systems.

REFERENCES/BIBLIOGRAPHY

- Abhik Banerjee, Abhirup Ghosh, Koustuvmoni Bharadwaj,” Mouse Control using a Web Camera based on Color Detection”, IJCTT, vol.9, Mar 2014
- Angel, Neethu.P.S, ”Real Time Static & Dynamic Hand Gesture Recognition”, International Journal of Scientific & Engineering Research Volume 4, Issue3, March-2013
- Chen-Chiung Hsieh and Dung-HuaLiou,” A Real Time Hand Gesture Recognition System Using Motion History Image”icsps, 2010
- Hojoon Park, “A Method for Controlling the Mouse Movement using a Real Time Camera”, Brown University, Providence, RI, USA, Department of computer science, 2008.
- Erdem, E. ardimci, Y. Atalay, V. Cetin, A. E. “Computer vision based mouse”, Acoustics, Speech, and Signal Processing, 2002. Proceedings.(ICASS). IEEE International Conference
- AbhirupGhosh, Abhik Banerjee “Mouse Control using a Web Camera based on Color Detection”
- J.L.Raheja, A.Chaudhary, K.Singal “Proposed using HSV algorithm but this uses special sensor is used to capture image and processes it” User has to spend more money for the sensor.
- Baohuaqiang, yijiezhai, mingliangzhou.” SqueezeNet and Fusion Network-Based Accurate Fast Fully Convolutional Network for Hand Detection and Gesture Recognition”. IEEE.2021.